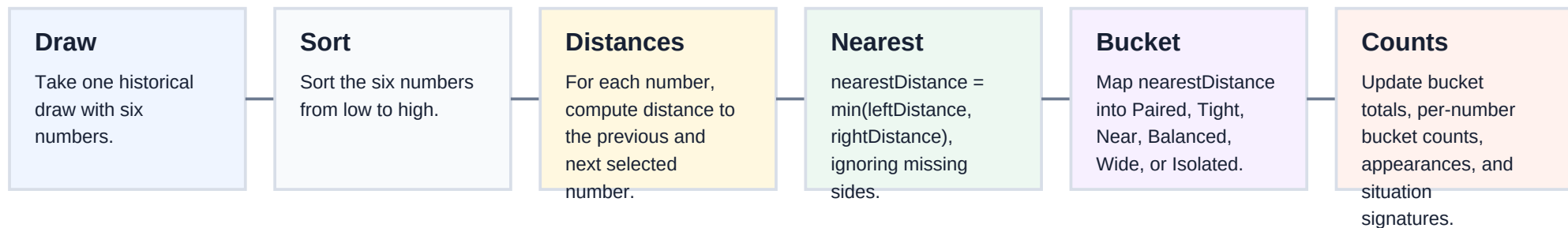


PyLotto Proximity Calculation

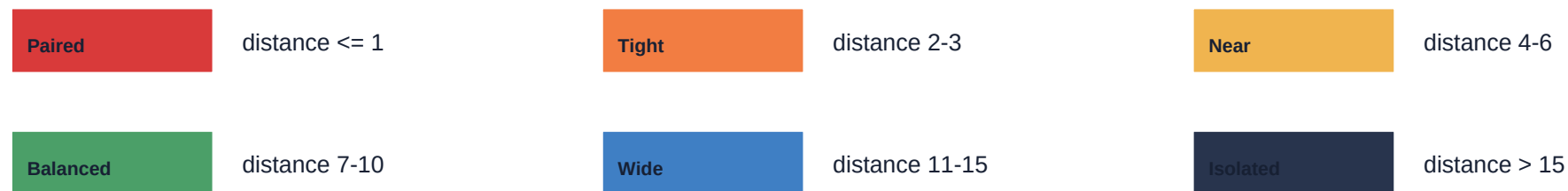
Flow diagram for nearest-neighbor spacing and number ranking



Nearest-neighbor example

If a draw is 4, 8, 9, 17, 31, 45, then number 8 has left distance 4 and right distance 1, so nearestDistance is 1 and the bucket is Paired. Number 31 has distances 14 and 14, so it is Wide.

Bucket rules



The proximity model describes the shape of actual drawn sets. It does not score unused numbers by their current board distance; it scores numbers by the historical sp

Proximity Ranking Formula

How bucket shares and per-number spacing histories become predictions

Historical counting

Bucket counts

For every drawn number, increment the bucket that matches its nearest-neighbor distance.

Distance totals

Add nearestDistance into that bucket to compute average spacing per bucket.

Number profiles

For each number, count how many times it appeared in each proximity bucket.

Summary formulas

Bucket share

$$\text{share}(\text{bucket}) = \text{bucketCount}(\text{bucket}) / (\text{drawCount} * 6)$$

Average nearest distance

$$\text{averageNearestDistance} = \text{bucketDistanceTotal} / \text{bucketCount}$$

Prediction formula for number n

For each bucket, multiply how often number n historically appeared in that bucket by the global share of that bucket. Sum those products, then divide by drawCount.

Score

$$\text{score}(n) = \text{sum over buckets} [\text{countForNumber}(n, \text{bucket}) * \text{bucketShare}(\text{bucket})] / \text{drawCount}$$

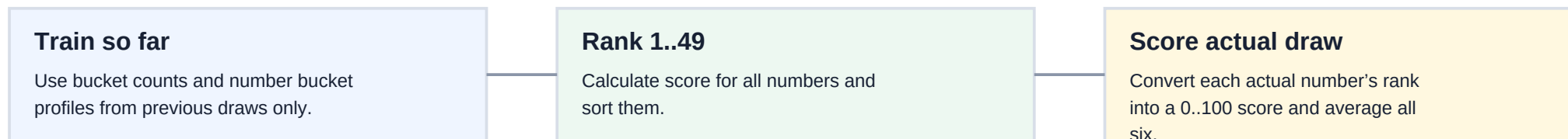
Sort order: score descending, then appearances descending, then smaller number.

Proximity Backtesting

How Draw Scores and Score Graphs evaluate proximity predictions

Previous-history-only evaluation

For each draw after the first, the app builds proximity predictions using only earlier draws. Then it compares the actual draw against that ranking.



Backtest metrics



The report also records situation signatures such as Paired x2 + Near x1 + Wide x3 to describe full-draw spacing patterns.